Create a Table with SQLModel - Use the Engine

Now let's get to the code. 👩‍💻

Make sure you are inside of your project directory and with your virtual environment activated as explained in Virtual Environments.

We will:

- Define a table with **SQLModel**
- Create the same SQLite database and table with **SQLModel**
- Use **DB Browser for SQLite** to confirm the operations

Here's a reminder of the table structure we want:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

Create the Table Model Class

The first thing we need to do is create a class to represent the data in the table.

A class like this that represents some data is commonly called a **model**.

> **Tip**
>
> That's why this package is called `SQLModel`. Because it's mainly used to create **SQL Models**.

For that, we will import `SQLModel` (plus other things we will also use) and create a class `Hero` that inherits from `SQLModel` and represents the **table model** for our heroes:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

This class `Hero` **represents the table** for our heroes. And each instance we create later will **represent a row** in the table.

We use the config `table=True` to tell **SQLModel** that this is a **table model**, it represents a table.

Info

It's also possible to have models without `table=True`, those would be only **data models**, without a table in the database, they would not be **table models**.

Those **data models** will be **very useful later**, but for now, we'll just keep adding the `table=True` configuration.

Define the Fields, Columns

The next step is to define the fields or columns of the class by using standard Python type annotations.

The name of each of these variables will be the name of the column in the table.

And the type of each of them will also be the type of table column:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

Let's now see with more detail these field/column declarations.

`None` Fields, Nullable Columns

Let's start with `age`, notice that it has a type of `int | None`.

That is the standard way to declare that something "could be an `int` or `None`" in Python.

And we also set the default value of `age` to `None`.

Python 3.10+

```
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

Tip

We also define `id` with `int | None`. But we will talk about `id` below.

Because the type is `int | None`:

- When validating data, `None` will be an allowed value for `age`.
- In the database, the column for `age` will be allowed to have `NULL` (the SQL equivalent to Python's `None`).

And because there's a default value `= None`:

- When validating data, this `age` field won't be required, it will be `None` by default.
- When saving to the database, the `age` column will have a `NULL` value by default.

Tip

The default value could have been something else, like `= 42`.

Primary Key `id`

Now let's review the `id` field. This is the **primary key** of the table.

So, we need to mark `id` as the **primary key**.

To do that, we use the special `Field` function from `sqlmodel` and set the argument `primary_key=True`:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

That way, we tell **SQLModel** that this `id` field/column is the primary key of the table.

But inside the SQL database, it is **always required** and can't be `NULL`. Why should we declare it with `int | None`?

The `id` will be required in the database, but it will be *generated by the database*, not by our code.

So, whenever we create an instance of this class (in the next chapters), we *will not set the* `id`. And the value of `id` will be `None` **until we save it in the database**, and then it will finally have a value.

```
my_hero = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")

do_something(my_hero.id)  # Oh no! my_hero.id is None! 😱🚨

# Imagine this saves it to the database
somehow_save_in_db(my_hero)

do_something(my_hero.id)  # Now my_hero.id has a value generated in DB 🎉
```

So, because in *our code* (not in the database) the value of `id` *could be* `None`, we use `int | None`. This way **the editor will be able to help us**, for example, if we try to access the `id` of an object that we haven't saved in the database yet and would still be `None`.

```
my_hero = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")

do_something(my_hero.id)
Argument 1 to "do_something" has incompatible type "Optional[int]";
expected "int" mypy(error)
Argument of type "int | None" cannot be assigned to parameter "id" of
type "int" in function "do_something"
  Type "int | None" cannot be assigned to type "int"
    Type "None" cannot be assigned to type
"int" Pylance(reportGeneralTypeIssues)
(variable) my_hero: Hero
View Problem (Alt+F8)    No quick fixes available
```

Now, because we are taking the place of the default value with our `Field()` function, we set **the actual default value** of `id` to `None` with the argument `default=None` in `Field()`:

```
Field(default=None)
```

If we didn't set the `default` value, whenever we use this model later to do data validation (powered by Pydantic) it would *accept* a value of `None` apart from an `int`, but it would still **require** passing that `None` value. And it would be confusing for whoever is using this model later (probably us), so **better set the default value here**.

Create the Engine

Now we need to create the SQLAlchemy **Engine**.

It is an object that handles the communication with the database.

If you have a server database (for example PostgreSQL or MySQL), the **engine** will hold the **network connections** to that database.

Creating the **engine** is very simple, just call `create_engine()` with a URL for the database to use:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

You should normally have a single **engine** object for your whole application and re-use it everywhere.

Tip

There's another related thing called a **Session** that normally should *not* be a single object per application.

But we will talk about it later.

Engine Database URL

Each supported database has its own URL type. For example, for **SQLite** it is `sqlite:///` followed by the file path. For example:

- `sqlite:///database.db`
- `sqlite:///databases/local/application.db`
- `sqlite:///db.sqlite`

SQLite supports a special database that lives all *in memory*. Hence, it's very fast, but be careful, the database gets deleted after the program terminates. You can specify this in-memory database by using just two slash characters (`//`) and no file name:

- `sqlite://`

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

You can read a lot more about all the databases supported by **SQLAlchemy** (and that way supported by **SQLModel**) in the SQLAlchemy documentation.

Engine Echo

In this example, we are also using the argument `echo=True`.

It will make the engine print all the SQL statements it executes, which can help you understand what's happening.

It is particularly useful for **learning** and **debugging**:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)
```

```
SQLModel.metadata.create_all(engine)
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

But in production, you would probably want to remove `echo=True`:

```
engine = create_engine(sqlite_url)
```

Engine Technical Details

Tip

If you didn't know about SQLAlchemy before and are just learning **SQLModel**, you can probably skip this section, scroll below.

You can read a lot more about the engine in the SQLAlchemy documentation.

SQLModel defines its own `create_engine()` function. It is the same as SQLAlchemy's `create_engine()`, but with the difference that it defaults to use `future=True` (which means that it uses the style of the latest SQLAlchemy, 1.4, and the future 2.0).

And SQLModel's version of `create_engine()` is type annotated internally, so your editor will be able to help you with autocompletion and inline errors.

Create the Database and Table

Now everything is in place to finally create the database and table:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"
```

```
engine = create_engine(sqlite_url, echo=True)

SQLModel.metadata.create_all(engine)
```

Tip

Creating the engine doesn't create the `database.db` file.

But once we run `SQLModel.metadata.create_all(engine)`, it creates the `database.db` file **and** creates the `hero` table in that database.

Both things are done in this single step.

Let's unwrap that:

```
SQLModel.metadata.create_all(engine)
```

SQLModel MetaData

The `SQLModel` class has a `metadata` attribute. It is an instance of a class `MetaData`.

Whenever you create a class that inherits from `SQLModel` **and is configured with `table = True`**, it is registered in this `metadata` attribute.

So, by the last line, `SQLModel.metadata` already has the `Hero` registered.

Calling `create_all()`

This `MetaData` object at `SQLModel.metadata` has a `create_all()` method.

It takes an **engine** and uses it to create the database and all the tables registered in this `MetaData` object.

SQLModel MetaData Order Matters

This also means that you have to call `SQLModel.metadata.create_all()` *after* the code that creates new model classes inheriting from `SQLModel`.

For example, let's imagine you do this:

- Create the models in one Python file `models.py`.
- Create the engine object in a file `db.py`.
- Create your main app and call `SQLModel.metadata.create_all()` in `app.py`.

If you only imported `SQLModel` and tried to call `SQLModel.metadata.create_all()` in `app.py`, it would not create your tables:

```
# This wouldn't work! 🚨
from sqlmodel import SQLModel

from .db import engine

SQLModel.metadata.create_all(engine)
```

It wouldn't work because when you import `SQLModel` alone, Python doesn't execute all the code creating the classes inheriting from it (in our example, the class `Hero`), so `SQLModel.metadata` is still empty.

But if you import the models *before* calling `SQLModel.metadata.create_all()`, it will work:

```
from sqlmodel import SQLModel
```

```
from . import models
from .db import engine

SQLModel.metadata.create_all(engine)
```

This would work because by importing the models, Python executes all the code creating the classes inheriting from `SQLModel` and registering them in the `SQLModel.metadata`.

As an alternative, you could import `SQLModel` and your models inside of `db.py`:

```
# db.py
from sqlmodel import SQLModel, create_engine
from . import models


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url)
```

And then import `SQLModel` *from* `db.py` in `app.py`, and there call `SQLModel.metadata.create_all()`:

```
# app.py
from .db import engine, SQLModel

SQLModel.metadata.create_all(engine)
```

The import of `SQLModel` from `db.py` would work because `SQLModel` is also imported in `db.py`.

And this trick would work correctly and create the tables in the database because by importing `SQLModel` from `db.py`, Python executes all the code creating the classes that inherit from `SQLModel` in that `db.py` file, for example, the class `Hero`.

Migrations

For this simple example, and for most of the **Tutorial - User Guide**, using `SQLModel.metadata.create_all()` is enough.

But for a production system you would probably want to use a system to migrate the database.

This would be useful and important, for example, whenever you add or remove a column, add a new table, change a type, etc.

But you will learn about migrations later in the Advanced User Guide.

Run The Program

Let's run the program to see it all working.

Put the code in a file `app.py` if you haven't already.

Python 3.10+

```
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)
```

```
SQLModel.metadata.create_all(engine)
```

Tip

Remember to activate the virtual environment before running it.

Now run the program with Python:

```bash
```

Info

I simplified the output above a bit to make it easier to read.

But in reality, instead of showing:

```
INFO Engine BEGIN (implicit)
```

it would show something like:

```
2021-07-25 21:37:39,175 INFO sqlalchemy.engine.Engine BEGIN (implicit)
```

`TEXT` or `VARCHAR`

In the example in the previous chapter we created the table using `TEXT` for some columns.

But in this output SQLAlchemy is using `VARCHAR` instead. Let's see what's going on.

Remember that each SQL Database has some different variations in what they support?

This is one of the differences. Each database supports some particular **data types**, like `INTEGER` and `TEXT`.

Some databases have some particular types that are special for certain things. For example, PostgreSQL and MySQL support `BOOLEAN` for values of `True` and `False`. SQLite accepts SQL with booleans, even when defining table columns, but what it actually uses internally are `INTEGER`s, with `1` to represent `True` and `0` to represent `False`.

The same way, there are several possible types for storing strings. SQLite uses the `TEXT` type. But other databases like PostgreSQL and MySQL use the `VARCHAR` type by default, and `VARCHAR` is one of the most common data types.

`VARCHAR` comes from **variable** length **character**.

SQLAlchemy generates the SQL statements to create tables using `VARCHAR`, and then SQLite receives them, and internally converts them to `TEXT`s.

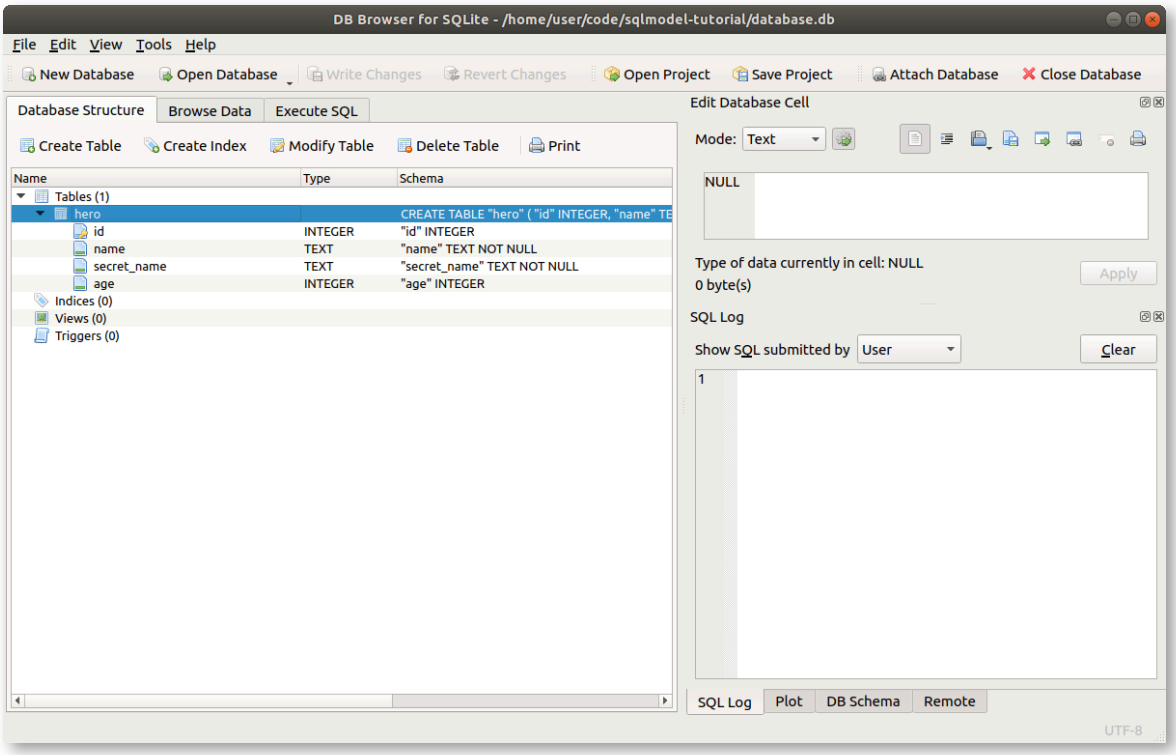
Additional to the difference between those two data types, some databases like MySQL require setting a maximum length for the `VARCHAR` types, for example `VARCHAR(255)` sets the maximum number of characters to 255.

To make it easier to start using **SQLModel** right away independent of the database you use (even with MySQL), and without any extra configurations, by default, `str` fields are interpreted as `VARCHAR` in most databases and `VARCHAR(255)` in MySQL, this way you know the same class will be compatible with the most popular databases without extra effort.

> **Tip**
>
> You will learn how to change the maximum length of string columns later in the Advanced Tutorial - User Guide.

Verify the Database

Now, open the database with **DB Browser for SQLite**, you will see that the program created the table `hero` just as before. 🎉



Refactor Data Creation

Now let's restructure the code a bit to make it easier to **reuse**, **share**, and **test** later.

Let's move the code that has the main **side effects**, that changes data (creates a file with a database and a table) to a function.

In this example it's just the `SQLModel.metadata.create_all(engine)`.

Let's put it in a function `create_db_and_tables()`:

Python 3.10+

```
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


if __name__ == "__main__":
    create_db_and_tables()
```

If `SQLModel.metadata.create_all(engine)` was not in a function and we tried to import something from this module (from this file) in another, it would try to create the database and table **every time** we executed that other file that imported this module.

We don't want that to happen like that, only when we **intend** it to happen, that's why we put it in a function, because we can make sure that the tables are created only when we call that function, and not when this module is imported somewhere else.

Now we would be able to, for example, import the `Hero` class in some other file without having those **side effects**.

> **Tip**
>
> 😅 **Spoiler alert**: The function is called `create_db_and_tables()` because we will have more **tables** in the future with other classes apart from `Hero`. 🚀

Create Data as a Script

We prevented the side effects when importing something from your `app.py` file.

But we still want it to **create the database and table** when we call it with Python directly as an independent script from the terminal, just as above.

> **Tip**
>
> Think of the word **script** and **program** as interchangeable.
>
> The word **script** often implies that the code could be run independently and easily. Or in some cases it refers to a relatively simple program.

For that we can use the special variable `__name__` in an `if` block:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


if __name__ == "__main__":
    create_db_and_tables()
```

About `__name__ == "__main__"`

The main purpose of the `__name__ == "__main__"` is to have some code that is executed when your file is called with:

```bash
```

...but is not called when another file imports it, like in:

```
from app import Hero
```

Tip

That `if` block using `if __name__ == "__main__":` is sometimes called the "**main block**".

The official name (in the Python docs) is "**Top-level script environment**".

More details

Let's say your file is named `myapp.py`.

If you run it with:

```bash
```

...then the internal variable `__name__` in your file, created automatically by Python, will have as value the string `"__main__"`.

So, the function in:

```
if __name__ == "__main__":
    create_db_and_tables()
```

...will run.

This won't happen if you import that module (file).

So, if you have another file `importer.py` with:

```
from myapp import Hero

# Some more code
```

...in that case, the automatic variable inside of `myapp.py` will not have the variable `__name__` with a value of `"__main__"`.

So, the line:

```
if __name__ == "__main__":
    create_db_and_tables()
```

...will **not** be executed.

Info

For more information, check the official Python docs.

Last Review

After those changes, you could run it again, and it would generate the same output as before.

But now we can import things from this module in other files.

Now, let's give the code a final look:

Python 3.10+

```python
from sqlmodel import Field, SQLModel, create_engine  ❷


class Hero(SQLModel, table=True):  ❸
    id: int | None = Field(default=None, primary_key=True)  ❹
    name: str  ❺
    secret_name: str  ❻
    age: int | None = None  ❼


sqlite_file_name = "database.db"  ❽
sqlite_url = f"sqlite:///{sqlite_file_name}"  ❾

engine = create_engine(sqlite_url, echo=True)  ❿


def create_db_and_tables():  ⓫
    SQLModel.metadata.create_all(engine)  ⓬


if __name__ == "__main__":  ⓭
    create_db_and_tables()  ⓮
```

❶ Import `Optional` from `typing` to declare fields that could be `None`.

❷ Import the things we will need from `sqlmodel`: `Field`, `SQLModel`, `create_engine`.

❸ Create the `Hero` model class, representing the `hero` table in the database.

And also mark this class as a **table model** with `table=True`.

❹ Create the `id` field:

It could be `None` until the database assigns a value to it, so we annotate it with `Optional` (`int | None` in Python 3.10+).

It is a **primary key**, so we use `Field()` and the argument `primary_key=True`.

❺ Create the `name` field.

It is required, so there's no default value, and it's not `Optional`.

❻ Create the `secret_name` field.

Also required.

❼ Create the `age` field.

It is not required, the default value is `None`.

In the database, the default value will be `NULL`, the SQL equivalent of `None`.

As this field could be `None` (and `NULL` in the database), we annotate it with `Optional` (`int | None` in Python 3.10+).

❽ Write the name of the database file.

❾ Use the name of the database file to create the database URL.

❿ Create the engine using the URL.

This doesn't create the database yet, no file or table is created at this point, only the **engine** object that will handle the connections with this specific database, and with specific support for SQLite (based on the URL).

⓫ Put the code that creates side effects in a function.

In this case, only one line that creates the database file with the table.

12 Create all the tables that were automatically registered in `SQLModel.metadata`.

13 Add a main block, or "Top-level script environment".

And put some logic to be executed when this is called directly with Python, as in:

```bash
```

...but that is not executed when importing something from this module, like:

```
from app import Hero
```

14 In this main block, call the function that creates the database file and the table.

This way when we call it with:

```bash
```

...it will create the database file and the table.

> **Tip**
>
> Review what each line does by clicking each number bubble in the code. 👆

Recap

We learnt how to use **SQLModel** to define how a table in the database should look like, and we created a database and a table using **SQLModel**.

We also refactored the code to make it easier to reuse, share, and test later.

In the next chapters we will see how **SQLModel** will help us interact with SQL databases from code. 🤓